



infocrumbs

Milestone project 4

Pep8 report



Accounts App

accounts/emails.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.core.mail import EmailMultiAlternatives
2 from django.template.loader import render_to_string
3 from django.conf import settings
4 from django.urls import reverse
5 from django.contrib.sites.models import Site
6
7
8 def send_security_alert_email(user, change_type, ip_address=None):
9     """
10     Sends a security alert email to the user.
11     """
12     subject = 'Important: Your InfoCrums Account Activity'
13     from_email = settings.DEFAULT_FROM_EMAIL
14     recipient_list = [user.email]
15
16     current_site = Site.objects.get_current()
17     site_domain = current_site.domain
18
19     context = {
20         'user': user,
21         'change_type': change_type,
22         'ip_address': ip_address if ip_address else 'Unknown',
23         'login_history_url': f'{settings.SITE_URL}{reverse("account_login")}',
24         'site_name': 'InfoCrums',
25         'site_domain': site_domain,
26     }
27
28     html_content = render_to_string(
29         'emails/security_alert.html', context
30     )
31     text_content = render_to_string(
32         'emails/security_alert.txt', context
33     )
34
35     msg = EmailMultiAlternatives(
36         subject,
37         text_content,
38         from_email,
39         recipient_list
40     )
41     msg.attach_alternative(html_content, "text/html")
42     msg.send()
```

Settings:



Results:

All clear, no errors found

accounts/forms.py

w: 1366px h: 599px



CI Python Linter

```
1 from django import forms
2 from allauth.account.forms import SignupForm
3 from django.contrib.auth import get_user_model
4 from .models import Profile, CustomUser
5
6
7 class CustomSignupForm(SignupForm):
8     """
9     Custom signup form to include additional fields like first name,
10     last name, and date of birth.
11     """
12     first_name = forms.CharField(
13         max_length=30,
14         label="First Name",
15         widget=forms.TextInput(attrs={'placeholder': 'First Name'})
16     )
17     last_name = forms.CharField(
18         max_length=30,
19         label="Last Name",
20         widget=forms.TextInput(attrs={'placeholder': 'Last Name'})
21     )
22     date_of_birth = forms.DateField(
23         label="Date of Birth",
24         widget=forms.DateInput(attrs={"type": "date"})
25     )
26
27 def save(self, request):
28     user = super().save(request)
29     user.first_name = self.cleaned_data["first_name"]
30     user.last_name = self.cleaned_data["last_name"]
31     user.save()
32
33     Profile.objects.update_or_create(
34         user=user,
35         defaults={
36             'date_of_birth': self.cleaned_data["date_of_birth"]
37         }
38     )
39
40     return user
41
42
```

Settings:



Results:

All clear, no errors found

accounts/models.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.conf import settings
2 from django.contrib.auth.models import AbstractUser
3 from django.db import models
4
5 from crumbs.models import Crumb
6 from feedback.models import Comment
7 from preferences.models import Topic
8
9
10 class CustomUser(AbstractUser):
11     email = models.EmailField(unique=True)
12     is_premium = models.BooleanField(default=False)
13     subscription_type = models.CharField(
14         max_length=20,
15         choices=[
16             ('none', 'None'),
17             ('weekly', 'Weekly'),
18             ('monthly', 'Monthly'),
19             ('annually', 'Annually'),
20         ],
21         default='none',
22     )
23
24     date_of_birth = models.DateField(null=True, blank=True)
25
26     def __str__(self):
27         return self.username
28
29
30 class Profile(models.Model):
31     user = models.OneToOneField(
32         settings.AUTH_USER_MODEL,
33         on_delete=models.CASCADE
34     )
35     saved_crumbs = models.ManyToManyField(Crumb, blank=True)
36     comment_history = models.ManyToManyField(Comment, blank=True)
37     topic_preferences = models.ManyToManyField(Topic, blank=True)
38
39     def __str__(self):
40         return f'{self.user.username}'s Profile'
41
```

Settings:



Results:

All clear, no errors found

accounts/signals.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.db.models.signals import pre_save, post_save
2 from django.dispatch import receiver
3 from django.contrib.auth import get_user_model
4 from django.conf import settings
5 from django.contrib.sites.models import Site
6 from django.contrib.auth.hashers import check_password
7
8 from .emails import send_security_alert_email
9 from .models import CustomUser, Profile
10
11
12 @receiver(post_save, sender=CustomUser)
13 def create_user_profile(sender, instance, created, **kwargs):
14     """
15     Creates a Profile instance for the user when a CustomUser is created.
16     This signal is triggered after a CustomUser instance is saved.
17     """
18     if created:
19         Profile.objects.create(user=instance)
20
21
22 @receiver(post_save, sender=CustomUser)
23 def save_user_profile(sender, instance, **kwargs):
24     """
25     Saves the Profile instance when a CustomUser is saved.
26     This signal is triggered after a CustomUser instance is saved.
27     """
28     instance.profile.save()
29
30
31 User = get_user_model()
32
33
34 # --- Signal for User Changes (Password & Profile Details) ---
35 @receiver(pre_save, sender=User)
36 def track_user_changes_pre_save(sender, instance, **kwargs):
37     """
38     Stores the original field values of a User instance
39     (including password hash)
40     before it's saved. This allows comparison in post_save.
41     """
42     if instance.pk:
```

Settings:



Results:

All clear, no errors found

accounts/tests.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from urllib import response
2 from django.test import TestCase, Client
3 from django.urls import reverse
4 from django.contrib.auth import get_user_model
5 from django.db.utils import IntegrityError
6 from django.core import mail # For testing emails at a later stage
7 from django.core.exceptions import ValidationError
8
9 # Import models from other apps that are linked to
10 # Profile for testing relationships
11 from accounts.models import Profile
12 from crumbs.models import Crumb
13 from feedback.models import Comment, SavedCrumb
14 from preferences.models import Topic, UserPreference
15 from subscriptions.models import (
16     SubscriptionPlan,
17     SubscriptionFrequency,
18     UserSubscription
19 )
20
21
22 # Get the custom user model as defined in settings.AUTH_USER_MODEL
23 CustomUser = get_user_model()
24
25
26 class CustomUserModelTest(TestCase):
27     """Test the CustomUser model and its methods."""
28     def test_user_creation(self):
29         """Test that a custom user can be created with default values."""
30         user = CustomUser.objects.create_user(
31             username='testuser',
32             email='test@example.com',
33             password='password123'
34         )
35         self.assertEqual(user.username, 'testuser')
36         self.assertEqual(user.email, 'test@example.com')
37         self.assertTrue(user.check_password('password123'))
38         self.assertTrue(user.is_active)
39         self.assertFalse(user.is_staff)
40         self.assertFalse(user.is_superuser)
41         self.assertEqual(user.is_premium, False)
42         self.assertEqual(user.subscription_type, 'none')
```

Settings:



Results:

All clear, no errors found

accounts/urls.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path("profile/", views.profile_view, name="account_profile"),
6     path(
7         'profile/account-details/',
8         views.load_account_details,
9         name='load_account_details_partial'
10    ),
11    path(
12        'account/update/',
13        views.account_update,
14        name='account_update'
15    ),
16    path(
17        'profile/saved-crums/',
18        views.load_saved_crums_partial,
19        name='load_saved_crums_partial'
20    ),
21    path(
22        'profile/comments/',
23        views.load_comments_partial,
24        name='load_comments_partial'
25    ),
26    path(
27        'profile/preferences/',
28        views.load_preferences_partial,
29        name='load_preferences_partial'
30    ),
31    path(
32        'profile/subscription/',
33        views.load_subscription_partial,
34        name='load_subscription_partial'
35    ),
36 ]
37
```

Settings:



Results:

All clear, no errors found

accounts/views.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from datetime import datetime
2 from django.http import JsonResponse
3 from django.utils import timezone
4 from django.contrib import messages
5 from django.shortcuts import render, redirect
6 from django.template.loader import render_to_string
7 from django.contrib.auth.decorators import login_required
8 from django.utils.timezone import now
9 from django.core.paginator import Paginator, EmptyPage, PageNotAnInteger
10
11 from accounts.forms import UserUpdateForm
12 from preferences.models import UserPreference
13 from feedback.models import SavedCrumb, Comment
14 from subscriptions.models import UserSubscription
15
16
17 @login_required
18 def profile_view(request):
19     """
20     Displays the user's profile with subscription details.
21     """
22     user = request.user
23     subscription = UserSubscription.objects.filter(
24         user=user,
25         active=True
26     ).first()
27     context = {
28         'user_subscription': subscription,
29     }
30     return render(request, 'account/profile.html', context)
31
32
33 @login_required
34 def load_account_details(request):
35     """
36     Loads the user's account details for AJAX requests.
37     """
38     if not request.user.is_authenticated:
39         return JsonResponse({'error': 'Authentication required.'}, status=401)
40
41     # Instantiate the form with the current user's data for initial display
42     form = UserUpdateForm(instance=request.user)
```

Settings:



Results:

All clear, no errors found



Checkout App

checkout/urls.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.urls import path
2 from . import views
3 from . import webhooks
4
5
6 urlpatterns = [
7     # Main checkout view, takes plan_id and frequency_id
8     path(
9         '<int:plan_id>/<int:frequency_id>/',
10        views.checkout_subscription,
11        name='checkout'
12    ),
13    # AJAX endpoint to cache checkout data in PaymentIntent metadata
14    path(
15        'cache_checkout_data/',
16        views.cache_checkout_data,
17        name='cache_checkout_data'
18    ),
19    # Stripe webhook endpoint
20    path('wh/', webhooks.webhook, name='webhook'),
21    # Success page after payment
22    path(
23        'success/<str:payment_intent_id>/',
24        views.checkout_success,
25        name='checkout_success'
26    ),
27 ]
28
```

Settings:



Results:

All clear, no errors found

checkout/views.py

w: 1366 px h: 599 px



CI Python Linter

```
1 import stripe
2 import json
3 from datetime import timedelta
4 from django.shortcuts import (
5     render,
6     redirect,
7     reverse,
8     get_object_or_404,
9     HttpResponseRedirect
10 )
11 from django.contrib import messages
12 from django.contrib.auth.decorators import login_required
13 from django.conf import settings
14 from django.views.decorators.http import require_POST
15 from django.utils.timezone import now
16
17 from subscriptions.models import (
18     SubscriptionPlan,
19     SubscriptionFrequency,
20     UserSubscription
21 )
22 from subscriptions.utils import calculate_subscription_price
23
24
25 # Set Stripe API key globally for the module
26 stripe.api_key = settings.STRIPE_SECRET_KEY
27
28
29 @login_required
30 def checkout_subscription(request, plan_id, frequency_id):
31     """
32     Display the checkout page for a selected subscription plan and frequency.
33
34     This view creates a Stripe PaymentIntent for the purchase and checks if
35     the user has an active subscription. If they do, and the new selection
36     differs from their current plan/frequency, it determines if the user is
37     eligible for an immediate switch (within 24 hours of original purchase)
38     or must wait until the current plan ends.
39     """
40     stripe_public_key = settings.STRIPE_PUBLIC_KEY
41
42     try:
```

Settings:



Results:

All clear, no errors found

checkout/webhook_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 import json
2 import stripe
3 from django.conf import settings
4 from django.http import HttpResponse
5 from django.utils.timezone import now
6 from django.contrib.auth import get_user_model
7 from datetime import timedelta
8
9 from subscriptions.email import send_subscription_confirmation_email
10 from subscriptions.models import (
11     SubscriptionPlan,
12     SubscriptionFrequency,
13     UserSubscription
14 )
15
16 User = get_user_model()
17
18
19 class StripeWH_Handler:
20     """Handle Stripe webhooks for subscriptions"""
21
22     def __init__(self, request):
23         self.request = request
24
25     def handle_event(self, event):
26         """
27         Handle a generic/unknown/unexpected webhook event
28         """
29         return HttpResponse(
30             content=f'Unhandled webhook received: {event["type"]}',
31             status=200
32         )
33
34     def handle_payment_intent_succeeded(self, event):
35         """
36         Handle the payment_intent.succeeded webhook from Stripe.
37         This is where the UserSubscription is created or updated.
38         """
39         intent = event.data.object
40         pid = intent.id # Stripe Payment Intent ID
41
42         # Extract metadata for subscription details
```

Settings:



Results:

All clear, no errors found

checkout/webhooks.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.http import HttpResponse
2 from django.conf import settings
3 from django.views.decorators.http import require_POST
4 from django.views.decorators.csrf import csrf_exempt
5
6 from checkout.webhook_handler import StripeWH_Handler
7
8 import stripe
9 import json
10
11
12 @require_POST
13 @csrf_exempt
14 def webhook(request):
15     """Listen for webhooks from Stripe"""
16     wh_secret = settings.STRIPE_WEBHOOK_SECRET
17     stripe.api_key = settings.STRIPE_SECRET_KEY
18
19     payload = request.body
20     sig_header = request.META['HTTP_STRIPE_SIGNATURE']
21     event = None
22
23     try:
24         event = stripe.Webhook.construct_event(
25             payload, sig_header, wh_secret
26         )
27     except ValueError as e:
28         # Invalid payload
29         return HttpResponse(status=400)
30     except stripe.error.SignatureVerificationError as e:
31         # Invalid signature
32         return HttpResponse(status=400)
33     except Exception as e:
34         return HttpResponse(content=e, status=400)
35
36     # Set up a webhook handler
37     handler = StripeWH_Handler(request)
38
39     # Map webhook events to relevant handler functions
40     event_map = {
41         "payment_intent.succeeded":
42             handler.handle_payment_intent_succeeded
```

Settings:



Results:

All clear, no errors found



Core App



CI Python Linter

```
1 from django import forms
2
3
4 class ContactForm(forms.Form):
5     """
6     Form for users to send messages to the site administrator.
7     Includes fields for name, email, subject, and message.
8     """
9     name = forms.CharField(
10         max_length=100,
11         widget=forms.TextInput(
12             attrs={'placeholder': 'Your Name', 'class': 'form-control'}
13         ),
14         help_text="Please enter your full name."
15     )
16     email = forms.EmailField(
17         widget=forms.EmailInput(
18             attrs={
19                 'placeholder': 'Your Email Address',
20                 'class': 'form-control'
21             }
22         ),
23         help_text="We'll use this to reply to you."
24     )
25     subject = forms.CharField(
26         max_length=200,
27         widget=forms.TextInput(attrs={
28             'placeholder': 'Subject of your message', 'class': 'form-control'
29         })
30     ),
31     help_text="A brief summary of your inquiry."
32 )
33 message = forms.CharField(
34     widget=forms.Textarea(
35         attrs={
36             'rows': 5, 'placeholder': 'Your message here...',
37             'class': 'form-control'
38         })
39     ),
40     help_text="Please provide detailed information so we can "
41     "assist you better.",
42     min_length=10 # Minimum characters for a message
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
79 response = self.client.get(reverse('login'))
80 self.assertEqual(response.status_code, 302) # Expect a redirect
81 self.assertRedirects(response, reverse('account_login'))
82
83 def test_logout_redirect(self):
84     """
85     Test that the /logout/ URL redirects through allauth logout
86     to the expected final page (e.g., home page).
87     """
88     response = self.client.get(reverse('logout'))
89     self.assertEqual(response.status_code, 302)
90     self.assertRedirects(
91         response,
92         reverse('account_logout'),
93         status_code=302,
94         target_status_code=302
95     )
96
97     response = self.client.get(reverse('logout'))
98
99     self.assertEqual(response.status_code, 302)
100
101     redirect_to_allauth_logout_url = response.url
102
103     self.assertEqual(
104         redirect_to_allauth_logout_url,
105         reverse('account_logout')
106     )
107
108     response_from_allauth_logout = self.client.get(
109         redirect_to_allauth_logout_url
110     )
111
112     self.assertEqual(response_from_allauth_logout.status_code, 302)
113
114     final_redirect_url = response_from_allauth_logout.url
115
116     self.assertEqual(final_redirect_url, reverse('home'))
117
118     final_page_response = self.client.get(final_redirect_url)
119     self.assertEqual(final_page_response.status_code, 200)
120
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
1 from django.urls import path
2 from django.shortcuts import redirect
3
4 from . import views
5
6 urlpatterns = [
7     path('', views.home, name='home'),
8     path('about/', views.about, name='about'),
9     path('faq/', views.faq_view, name='faq'),
10    path('contact/', views.contact_view, name='contact'),
11    # Authentication URLs
12    path('signup/', lambda request: redirect('account_signup'), name='signup'),
13    path('login/', lambda request: redirect('account_login'), name='login'),
14    path('logout/', lambda request: redirect('account_logout'), name='logout'),
15 ]
```

Settings:



Results:

All clear, no errors found

core/views.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.shortcuts import render, redirect
2 from django.contrib import messages
3 from django.core.mail import EmailMultiAlternatives
4 from django.template.loader import render_to_string
5 from django.conf import settings
6 from django.utils import timezone
7 from django.contrib.sites.models import Site
8
9 from subscriptions.models import UserSubscription
10 from .forms import ContactForm
11
12
13 # home
14 def home(request):
15     """
16     Renders the home page with dynamic call-to-action based on user's
17     subscription status.
18     """
19     user_subscription_status = {
20         'is_subscribed': False,
21         'is_premium': False,
22         'has_active_sub_plan': None # Stores the name of active plan if any
23     }
24
25     if request.user.is_authenticated:
26         # Get the latest active subscription for the current user
27         active_subscription = UserSubscription.objects.filter(
28             user=request.user,
29             active=True,
30             end_date__gte=timezone.now()
31         ).order_by('-end_date').first()
32
33         if active_subscription:
34             user_subscription_status['is_subscribed'] = True
35             user_subscription_status['has_active_sub_plan'] = (
36                 active_subscription.plan.name
37             )
38
39         if active_subscription.plan.name.lower() == 'premium':
40             user_subscription_status['is_premium'] = True
41
42     context = {
```

Settings:



Results:

All clear, no errors found



Crumbs App



CI Python Linter

```
1 from django.db import models
2 from taggit.managers import TaggableManager
3
4 from preferences.models import Topic
5
6
7 class Crumb(models.Model):
8     title = models.CharField(max_length=1000)
9     summary = models.TextField()
10    url = models.URLField(max_length=2000, unique=True)
11    source = models.CharField(max_length=500)
12    topic = models.ForeignKey(Topic, on_delete=models.CASCADE)
13    tags = TaggableManager(blank=True)
14    published_at = models.DateTimeField()
15    added_on = models.DateTimeField(auto_now_add=True)
16
17    def __str__(self):
18        return self.title
19
20
```

Settings:



Results:

All clear, no errors found

crumbs/tests.py

w: 1366px h: 599px



CI Python Linter

```
1 import datetime
2
3 from django.test import TestCase, Client
4 from django.urls import reverse
5 from django.utils import timezone
6
7
8 from accounts.models import CustomUser, Profile
9 from preferences.models import Topic, UserPreference
10 from subscriptions.models import SubscriptionPlan, UserSubscription
11 from feedback.models import SavedCrumb
12 from .models import Crumb
13
14
15 class CrumbModelTest(TestCase):
16     """
17     Tests for the Crumb model.
18     """
19
20     def setUp(self):
21         """
22         Set up common data for Crumb model tests.
23         """
24         self.unique_topic_name = 'TestTopicForCrumbModel'
25         self.topic = Topic.objects.create(name=self.unique_topic_name)
26
27     def test_crumb_creation(self):
28         """
29         Ensure a Crumb can be created and its fields are set correctly.
30         """
31         crumb = Crumb.objects.create(
32             title='Test Crumb Title',
33             summary='This is a summary for the test crumb.',
34             url='http://example.com/test',
35             source='Test Source',
36             topic=self.topic,
37             published_at=timezone.now() - datetime.timedelta(days=1)
38         )
39         self.assertEqual(crumb.title, 'Test Crumb Title')
40         self.assertEqual(crumb.topic, self.topic)
41         self.assertIsNotNone(crumb.added_on)
42         self.assertLess(crumb.published_at, crumb.added_on)
```

Settings:



Results:

All clear, no errors found

crumbs/urls.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('crumb_list/', views.crumb_list, name='crumb_list'),
6     path('<int:pk>/', views.crumb_detail, name='crumb_detail'),
7 ]
```

Settings:



Results:

All clear, no errors found

crumbs/views.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils import timezone
2 from django.shortcuts import render, get_object_or_404, redirect
3 from django.core.paginator import Paginator
4
5 from preferences.models import UserPreference, Topic
6 from subscriptions.models import UsersSubscription
7 from feedback.forms import CommentForm
8 from feedback.models import Comment, SavedCrumb
9 from .models import Crumb
10
11
12 def crumb_list(request):
13     """
14     View to list crumbs for authenticated and subscribed users.
15     Filters crumbs by user preferences and subscription plan,
16     with optional topic filtering via GET parameter.
17     """
18     user = request.user
19     selected_topic = request.GET.get('topic')
20     crumbs = Crumb.objects.none()
21     saved_crums = []
22     topics = Topic.objects.all()
23
24     # Enforce login
25     if not user.is_authenticated:
26         return redirect('account_login')
27
28     # Enforce active subscription
29     subscription = UsersSubscription.objects.filter(
30         user=user, active=True
31     ).first()
32     if not subscription:
33         return redirect('choose_plan')
34
35     # Get user preferences
36     pref_obj = UserPreference.objects.filter(user=user).first()
37     preferred_topics = pref_obj.topics.values_list(
38         'id', flat=True
39     ) if pref_obj else []
40
41     # Limit topics for basic plan
42     if subscription.plan and subscription.plan.name.lower() == "basic":
```

Settings:



Results:

All clear, no errors found



Feedback App

feedback/forms.py

w: 1366px h: 599px



CI Python Linter

```
1 from django import forms
2 from .models import Comment
3
4
5 class CommentForm(forms.ModelForm):
6     """
7     Form for adding a comment to a Crumb.
8     This form includes a single field for the comment content with
9     min/max length validation.
10    """
11    # Define the comment_body field directly with min/max length
12    comment_body = forms.CharField(
13        label="",
14        min_length=5,
15        max_length=500,
16        widget=forms.Textarea(attrs={
17            'rows': 3,
18            'placeholder': 'Write your comment here ...',
19            'class': 'form-control'
20        }),
21        help_text="Your comment should be between 5 and 500 characters."
22    )
23
24 class Meta:
25     """
26     Meta class to define the model and fields for the form.
27     We use an empty fields list to indicate that we are overriding the
28     default fields with our custom field.
29     """
30     model = Comment
31     fields = []
32
33 def save(self, commit=True):
34     comment = super().save(commit=False)
35     comment.content = self.cleaned_data['comment_body']
36     if commit:
37         comment.save()
38     return comment
39
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
1 from django.db import models
2 from django.conf import settings
3
4 from crumbs.models import Crumb
5
6
7 class SavedCrumb(models.Model):
8     """
9     Model representing a saved Crumb by a user.
10    """
11    user = models.ForeignKey(
12        settings.AUTH_USER_MODEL,
13        on_delete=models.CASCADE,
14        related_name='saved_crumbs'
15    )
16    crumb = models.ForeignKey(
17        Crumb, on_delete=models.CASCADE, related_name='saves'
18    )
19    saved_at = models.DateTimeField(auto_now_add=True)
20
21 class Meta:
22     constraints = [
23         models.UniqueConstraint(
24             fields=['user', 'crumb'], name='unique_saved_crumb'
25         )
26     ]
27
28 def __str__(self):
29     return f"{self.user.username} saved {self.crumb.title}"
30
31
32 class LikedCrumb(models.Model):
33     """
34     Model representing a like on a crumb.
35    """
36    user = models.ForeignKey(
37        settings.AUTH_USER_MODEL,
38        on_delete=models.CASCADE,
39        related_name='liked_crumbs'
40    )
41    crumb = models.ForeignKey(
42        Crumb, on_delete=models.CASCADE, related_name='likes'
```

Settings:



Results:

All clear, no errors found

feedback/tests.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.test import TestCase, Client
2 from django.urls import reverse
3 from django.utils import timezone
4 from datetime import date
5 from django.db.utils import IntegrityError
6
7 from accounts.models import CustomUser
8 from crumbs.models import Crumb, Topic
9 from .models import SavedCrumb, LikedCrumb, Comment
10 from .forms import CommentForm
11
12
13 class FeedbackModelsTest(TestCase):
14     """
15     Test cases for SavedCrumb, LikedCrumb, and Comment models.
16     """
17     def setUp(self):
18         """
19         Set up test data for SavedCrumb, LikedCrumb, and Comment models.
20         """
21         self.user = CustomUser.objects.create_user(
22             username='testuser',
23             email='test@example.com',
24             password='password123',
25             date_of_birth=date(1990, 1, 1)
26         )
27         self.topic = Topic.objects.create(name='Technology')
28         self.crumb = Crumb.objects.create(
29             title='Test Crumb',
30             summary='This is a summary for the test crumb.',
31             published_at=timezone.now(),
32             topic=self.topic,
33             url='http://example.com/test-crumb-model-1'
34         )
35
36     def test_saved_crumb_creation(self):
37         """
38         Test that a SavedCrumb can be created and has the correct fields.
39         """
40         saved_crumb = SavedCrumb.objects.create(
41             user=self.user,
42             crumb=self.crumb
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('add/<int:crumb_id>/', views.add_comment, name='add_comment'),
6     path('edit/<int:comment_id>/', views.edit_comment, name='edit_comment'),
7     path(
8         'delete/<int:comment_id>/',
9         views.delete_comment, name='delete_comment'
10    ),
11    path(
12        'crumb/save/<int:crumb_id>/',
13        views.toggle_save_crumb,
14        name='toggle_save_crumb'
15    ),
16 ]
17
```

Settings:



Results:

All clear, no errors found

feedback/views.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.shortcuts import render, get_object_or_404, redirect
2 from django.http import HttpResponseRedirect, JsonResponse
3 from django.contrib.auth.decorators import login_required
4 from django.contrib import messages
5
6 from .models import Comment, SavedCrumb
7 from .forms import CommentForm
8 from crumbs.models import Crumb
9
10
11 @login_required
12 def add_comment(request, crumb_id):
13     """
14     Add a comment to a Crumb.
15     This view handles both GET and POST requests. For GET requests,
16     it returns a form for adding a comment. For POST requests,
17     it processes the form submission and saves the comment.
18     If the request is an AJAX request, it returns a JSON response
19     with the comment details. Otherwise, it redirects to the crumb detail page.
20     """
21     crumb = get_object_or_404(Crumb, id=crumb_id)
22
23     if request.method == "POST" and request.user.is_authenticated:
24         form = CommentForm(request.POST)
25         if form.is_valid():
26             comment = form.save(commit=False)
27             comment.crumb = crumb
28             comment.user = request.user
29             comment.save()
30
31             if request.headers.get("x-requested-with") == "XMLHttpRequest":
32                 comments = crumb.comments.order_by("-created_at")
33                 return render(request, "feedback/includes/comment_list.html", {
34                     "comments": comments,
35                     "user": request.user
36                 })
37             return redirect("crumb_detail", pk=crumb.id)
38
39     return HttpResponseRedirect("Invalid comment or unauthenticated.")
40
41
42
```

Settings:



Results:

All clear, no errors found



Pipeline App

pipeline/utils.py

w: 1366 px h: 599 px



CI Python Linter

```
1 import requests
2 import json
3 from django.conf import settings
4
5
6 # Define constants for Hugging Face API
7 HUGGINGFACE_API_URL = (
8     "https://api-inference.huggingface.co/models/facebook/bart-large-cnn"
9 )
10 # Max characters to send for summarization. Adjust as needed.
11 MAX_SUMMARY_INPUT_LENGTH = 1000
12 # Timeout for the API request in seconds
13 HF_API_TIMEOUT = 90
14
15
16 def summarize_text(text):
17     """
18     Sends text to Hugging Face API for summarization.
19     Truncates text to MAX_SUMMARY_INPUT_LENGTH before sending.
20     Returns an empty string if summarization fails or times out.
21     """
22     if not text:
23         return ""
24
25     # Ensure API key is set
26     api_key = getattr(settings, 'HF_API_TOKEN', None)
27     if not api_key:
28         print("HuggingFace API key is not set in Django settings.")
29         return ""
30
31     headers = {"Authorization": f"Bearer {api_key}"}
32
33     # Truncate text to avoid excessively long inputs
34     truncated_text = text[:MAX_SUMMARY_INPUT_LENGTH]
35
36     payload = {"inputs": truncated_text}
37
38     try:
39         response = requests.post(
40             HUGGINGFACE_API_URL,
41             headers=headers,
42             json=payload,
```

Settings:



Results:

All clear, no errors found



Pipeline App

Handlers folder

pipeline/handlers/cars_transport_handler.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_cars_transport_crumbs(crumb_data_list):
12     """
13     Handles cars and transport related crumbs by creating or updating
14     crumb objects. Each crumb is primarily associated with the
15     'cars-and-transport' topic and can receive additional tags
16     based on its content.
17
18     Args:
19         crumb_data_list (list): List of dictionaries containing crumb data.
20
21     Returns:
22         int: Number of Crumb objects created.
23     """
24     cars_transport_slug = "cars-transport"
25     try:
26         cars_transport_topic = Topic.objects.get(slug=cars_transport_slug)
27     except Topic.DoesNotExist:
28         # Create the topic if it doesn't exist
29         cars_transport_topic = Topic.objects.create(
30             name="cars & transport",
31             slug=cars_transport_slug
32         )
33
34     created_count = 0
35     for item in crumb_data_list:
36         title = item.get("title", "")[:255]
37         url = item.get("url")
38         raw_summary = item.get("summary", "")
39
40         if not title or not url:
41             print(f"Skipping cars and transport crumb due to missing title "
42                   f"or URL: {item}")
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/diy_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_diy_crumbs(crumb_data_list):
12     """
13     Handles DIY (Do-It-Yourself) related crumbs by creating or updating
14     crumb objects. Each crumb is primarily associated with the 'diy' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     diy_slug = "diy"
24     try:
25         diy_topic = Topic.objects.get(slug=diy_slug)
26     except Topic.DoesNotExist:
27         # Create the topic if it doesn't exist
28         diy_topic = Topic.objects.create(
29             name="DIY",
30             slug=diy_slug
31         )
32
33     created_count = 0
34     for item in crumb_data_list:
35         title = item.get("title", "")[:255]
36         url = item.get("url")
37         raw_summary = item.get("summary", "")
38
39         if not title or not url:
40             print(f"Skipping DIY crumb due to missing title or URL: {item}")
41             continue
42
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/environment_handler.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils.dateparse import parse_datetime
2 from django.utils.text import slugify
3 from django.utils.timezone import now as timezone_now
4 from django.utils import timezone
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import summarize_text, clean_text, tag_crumb_text
9
10
11 def handle_environment_articles(articles):
12     """
13     Handles environment-related articles by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'environment' topic and can
15     receive additional tags based on its content.
16
17     :param articles: List of dictionaries containing article data from
18     NewsData.io.
19     :return: Number of Crumb objects created.
20     """
21     environment_slug = "environment"
22     try:
23         environment_topic = Topic.objects.get(slug=environment_slug)
24     except Topic.DoesNotExist:
25         # Create the topic if it doesn't exist
26         environment_topic = Topic.objects.create(
27             name="environment",
28             slug=environment_slug
29         )
30
31     created_count = 0
32     for article in articles:
33         title = article.get("title", "")[:255]
34         url = article.get("link")
35         raw_description = article.get("description", "")
36
37         if not title or not url:
38             print(
39                 f"Skipping environment article due to missing title or "
40                 f"URL: {article}"
41             )
42             continue
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/fashion_handler.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_fashion_crumbs(crumb_data_list):
12     """
13     Handles fashion-related crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'fashion' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     fashion_slug = "fashion"
24     try:
25         fashion_topic = Topic.objects.get(slug=fashion_slug)
26     except Topic.DoesNotExist:
27         fashion_topic = Topic.objects.create(
28             name="fashion",
29             slug=fashion_slug
30         )
31
32     created_count = 0
33     for item in crumb_data_list:
34         title = item.get("title", "")[:255]
35         url = item.get("url")
36         raw_summary = item.get("summary", "")
37
38         if not title or not url:
39             print(f"Skipping fashion crumb due to missing title or URL: "
40                 f"{item}")
41             continue
42
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/finance_handler.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils.dateparse import parse_datetime
2 from django.utils.text import slugify
3 from django.utils.timezone import now as timezone_now
4 from django.utils import timezone
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import summarize_text, clean_text, tag_crumb_text
9
10
11 def handle_finance_crumbs(crumb_data_list):
12     """
13     Handles finance-related crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'stock-crypto-finance' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     finance_slug = "stock-crypto-finance"
24     try:
25         finance_topic = Topic.objects.get(slug=finance_slug)
26     except Topic.DoesNotExist:
27         finance_topic = Topic.objects.create(
28             name="stock, crypto & finance",
29             slug=finance_slug
30         )
31
32     created_count = 0
33     for item in crumb_data_list:
34         title = item.get("title", "")[:255]
35         url = item.get("url")
36         raw_summary = item.get("summary", "")
37
38         if not title or not url:
39             print(
40                 f"Skipping finance crumb due to missing title or URL: {item}"
41             )
42             continue
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/food_handler.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_food_drink_crumbs(crumb_data_list):
12     """
13     Handles food and drink related crumbs by creating or updating Crumb
14     objects.
15     Each crumb is primarily associated with the 'food-and-drink' topic
16     and can receive additional tags based on its content.
17
18     Args:
19         crumb_data_list (list): List of dictionaries containing crumb data.
20
21     Returns:
22         int: Number of Crumb objects created.
23     """
24     food_drink_slug = "food-and-drink"
25     try:
26         food_drink_topic = Topic.objects.get(slug=food_drink_slug)
27     except Topic.DoesNotExist:
28         # Create the topic if it doesn't exist
29         food_drink_topic = Topic.objects.create(
30             name="Food and drink",
31             slug=food_drink_slug
32         )
33
34     created_count = 0
35     for item in crumb_data_list:
36         title = item.get("title", "")[:255]
37         url = item.get("url")
38         raw_summary = item.get("summary", "")
39
40         if not title or not url:
41             print(f"Skipping food and drink crumb due to missing title or "
42                   f"url: {item}")
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/music_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_music_crumbs(crumb_data_list):
12     """
13     Handles music-related crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'music' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     music_slug = "music"
24     try:
25         music_topic = Topic.objects.get(slug=music_slug)
26     except Topic.DoesNotExist:
27         # Create the topic if it doesn't exist
28         music_topic = Topic.objects.create(
29             name="music",
30             slug=music_slug
31         )
32
33     created_count = 0
34     for item in crumb_data_list:
35         title = item.get("title", "")[:255]
36         url = item.get("url")
37         raw_summary = item.get("summary", "")
38
39         if not title or not url:
40             print(f"Skipping music crumb due to missing title or URL: {item}")
41             continue
42
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/news_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_world_news_crumbs(crumb_data_list):
12     """
13     Handles world news crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'world-news' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     world_news_slug = "world-news"
24     try:
25         world_news_topic = Topic.objects.get(slug=world_news_slug)
26     except Topic.DoesNotExist:
27         # Create the topic if it doesn't exist
28         world_news_topic = Topic.objects.create(
29             name="world news",
30             slug=world_news_slug
31         )
32
33     created_count = 0
34     for item in crumb_data_list:
35         title = item.get("title", "")[:255]
36         url = item.get("url")
37         raw_summary = item.get("summary") or item.get("description", "")
38
39         if not title or not url:
40             print(f"Skipping world news crumb due to missing title or URL: "
41                   f"{item}")
42             continue
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/plants_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.utils.dateparse import parse_datetime
2 from django.utils.text import slugify
3 from django.utils.timezone import now
4
5 from crumbs.models import Crumb
6 from preferences.models import Topic
7 from pipeline.utils import summarize_text, clean_text, tag_crumb_text
8
9
10 def handle_plant_data(source, data_list):
11     """
12     Handles plants and gardening crumbs by creating or updating Crumb objects
13     based on the provided crumb data list. Each crumb is primarily associated
14     with the 'plants-and-gardening' topic and can receive additional tags
15     based on its content.
16
17     :param source: The original source name (e.g., "Perennial", "Trefle").
18     :param data_list: List of dictionaries containing crumb data from a
19     fetcher.
20     :return: Number of Crumb objects created.
21     """
22     # Ensure the primary 'plants-and-gardening' topic exists
23     plants_slug = "plants-and-gardening"
24     try:
25         plant_topic = Topic.objects.get(slug=plants_slug)
26     except Topic.DoesNotExist:
27         plant_topic = Topic.objects.create(
28             name="plants and gardening",
29             slug=plants_slug
30         )
31
32     created_count = 0
33     for item in data_list:
34         # Basic validation for essential fields
35         if not item.get("title") or not item.get("url"):
36             print(f"Skipping crumb from {source} due to missing title or URL.")
37             continue
38
39         # Check for existing crumb by title and URL to prevent duplicates
40         if Crumb.objects.filter(title=item["title"], url=item["url"]).exists():
41             continue
42
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/sports_handler.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.utils.dateparse import parse_datetime
2 from django.utils.text import slugify
3 from django.utils import timezone
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import summarize_text, clean_text, tag_crumb_text
9
10
11 def handle_sports_crumbs(crumb_data_list):
12     """
13     Handles sports-related crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'sports-and-fitness' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     sports_slug = "sports-and-fitness"
24     try:
25         sports_topic = Topic.objects.get(slug=sports_slug)
26     except Topic.DoesNotExist:
27         sports_topic = Topic.objects.create(
28             name="sports and fitness",
29             slug=sports_slug
30         )
31
32     created_count = 0
33     for item in crumb_data_list:
34         title = item.get("title", "")[:255]
35         url = item.get("url")
36         raw_summary = item.get("summary", "")
37
38         if not title or not url:
39             print(f"Skipping sports crumb due to missing title or URL: {item}")
40             continue
41
42         if Crumb.objects.filter(title=title, url=url).exists():
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/technology_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_technology_crumbs(crumb_data_list):
12     """
13     Handles technology-related crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'technology' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     technology_slug = "technology"
24     try:
25         technology_topic = Topic.objects.get(slug=technology_slug)
26     except Topic.DoesNotExist:
27         # Create the topic if it doesn't exist
28         technology_topic = Topic.objects.create(
29             name="technology",
30             slug=technology_slug
31         )
32
33     created_count = 0
34     for item in crumb_data_list:
35         title = item.get("title", "")[:255]
36         url = item.get("url")
37         raw_summary = item.get("summary", "")
38
39         if not title or not url:
40             print(f"Skipping technology crumb due to missing title or URL: "
41                   f"{item}")
42             continue
```

Settings:



Results:

All clear, no errors found

pipeline/handlers/trivia_handler.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.utils import timezone
2 from django.utils.dateparse import parse_datetime
3 from django.utils.text import slugify
4 from django.utils.timezone import now as timezone_now
5
6 from crumbs.models import Crumb
7 from preferences.models import Topic
8 from pipeline.utils import clean_text, summarize_text, tag_crumb_text
9
10
11 def handle_trivia_fun_crumbs(crumb_data_list):
12     """
13     Handles trivia and fun crumbs by creating or updating Crumb objects.
14     Each crumb is primarily associated with the 'trivia-and-fun' topic
15     and can receive additional tags based on its content.
16
17     Args:
18         crumb_data_list (list): List of dictionaries containing crumb data.
19
20     Returns:
21         int: Number of Crumb objects created.
22     """
23     trivia_fun_slug = "trivia-and-fun"
24     try:
25         trivia_fun_topic = Topic.objects.get(slug=trivia_fun_slug)
26     except Topic.DoesNotExist:
27         # Create the topic if it doesn't exist
28         trivia_fun_topic = Topic.objects.create(
29             name="trivia and fun",
30             slug=trivia_fun_slug
31         )
32
33     created_count = 0
34     for item in crumb_data_list:
35         title = item.get("title", "")[:255]
36         url = item.get("url")
37         raw_summary = item.get("summary", "")
38
39         if not title or not url:
40             print(f"Skipping trivia/fun crumb due to missing title or URL: "
41                   f"{item}")
42             continue
```

Settings:



Results:

All clear, no errors found



Pipeline App

Management Command
folder

pipeline/management/command/fetch_crumbs.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.core.management.base import BaseCommand
2
3 from pipeline.tasks import (
4     plants,
5     environment,
6     sports,
7     finance,
8     news,
9     music,
10    technology,
11    food,
12    trivia,
13    fashion,
14    cars_transport,
15    diy,
16 )
17 from pipeline.handlers import (
18     plants_handler,
19     environment_handler,
20     sports_handler,
21     finance_handler,
22     news_handler,
23     music_handler,
24     technology_handler,
25     food_handler,
26     trivia_handler,
27     fashion_handler,
28     cars_transport_handler,
29     diy_handler,
30 )
31
32
33 class Command(BaseCommand):
34     help = 'Fetches crumbs from various sources and adds them to the database.'
35
36     def handle(self, *args, **options):
37         self.stdout.write("Starting crumb fetching process...")
38         total_created = 0
39
40         # Fetches and handles plants and gardening crumbs from 3 APIs
41         self.stdout.write("Fetching plant-related info...")
42
```

Settings:



Results:

All clear, no errors found



Pipeline App

Tasks folder

pipeline/tasks/cars_transport.py

w: 1366 px h: 599 px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_newsapi_cars_transport_news():
7     """
8     Fetches cars and transport news articles from NewsAPI.org API
9     using the /v2/everything endpoint.
10
11     Returns:
12         list: A list of dictionaries containing article details.
13     """
14     crumbs = []
15     try:
16         url = getattr(settings, 'NEWSAPI_CARS_TRANSPORT_URL', None)
17         if not url:
18             raise ValueError(
19                 "NEWSAPI_CARS_TRANSPORT_URL is not set in Django settings.")
20
21         response = requests.get(url, timeout=15)
22         response.raise_for_status()
23         data = response.json()
24
25         # NewsAPI.org returns results under the 'articles' key
26         for article in data.get("articles", []):
27             title = article.get("title")
28             url = article.get("url")
29             summary = article.get("description")
30             source_name = article.get("source", {}).get("name")
31             published_at_str = article.get("publishedAt")
32
33             if not title or not url:
34                 continue
35
36             crumbs.append(
37                 {
38                     "title": title,
39                     "summary": summary,
40                     "url": url,
41                     "source": source_name if source_name else
42                         "NewsAPI.org Cars & Transport"
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/diy.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_newsapi_diy_news():
7     """
8     Fetches DIY (Do-It-Yourself) and home improvement news articles
9     from NewsAPI.org API using the /v2/everything endpoint.
10
11     Returns:
12         list: A list of dictionaries containing article details.
13     """
14     crumbs = []
15     try:
16         url = getattr(settings, 'NEWSAPI_DIY_URL', None)
17         if not url:
18             raise ValueError(
19                 "NEWSAPI_DIY_URL is not set in Django settings.")
20
21         response = requests.get(url, timeout=15)
22         response.raise_for_status()
23         data = response.json()
24
25         # NewsAPI.org returns results under the 'articles' key
26         for article in data.get("articles", []):
27             title = article.get("title")
28             url = article.get("url")
29             summary = article.get("description")
30             source_name = article.get("source", {}).get("name")
31             published_at_str = article.get("publishedAt")
32
33             if not title or not url:
34                 continue
35
36             crumbs.append({
37                 "title": title,
38                 "summary": summary,
39                 "url": url,
40                 "source": source_name if source_name else "NewsAPI.org DIY",
41                 "published_at": published_at_str,
42             })
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/environment.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3
4
5 def fetch_environment_news():
6     """
7     Fetches environment-related news articles from NewsData.io API.
8     """
9     try:
10         url = getattr(settings, 'NEWSDATA_API_URL', None)
11         if not url:
12             raise ValueError("NEWSDATA_API_URL is not set in Django settings.")
13
14         response = requests.get(url, timeout=15)
15         response.raise_for_status()
16         data = response.json()
17
18         return data.get("results", [])
19     except requests.exceptions.RequestException as req_err:
20         print(f"NewsData.io API request error: {req_err}")
21         return []
22     except ValueError as val_err:
23         print(f"Configuration error for NewsData.io fetch: {val_err}")
24         return []
25     except Exception as e:
26         print(f"NewsData.io unexpected error: {e}")
27         return []
28
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/fashion.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_newsapi_fashion_news():
7     """
8     Fetches fashion news articles from NewsAPI.org API using the /everything
9     endpoint.
10
11     Returns:
12         list: A list of dictionaries containing article details.
13     """
14     crumbs = []
15     try:
16         url = getattr(settings, 'NEWSAPI_FASHION_URL', None)
17         if not url:
18             raise ValueError(
19                 "NEWSAPI_FASHION_URL is not set in Django settings.")
20
21         response = requests.get(url, timeout=15)
22         response.raise_for_status()
23         data = response.json()
24
25         # NewsAPI.org returns results under the 'articles' key
26         for article in data.get("articles", []):
27             title = article.get("title")
28             url = article.get("url")
29             summary = article.get("description")
30             source_name = article.get("source", {}).get("name")
31             published_at_str = article.get("publishedAt")
32
33             if not title or not url:
34                 continue
35
36             crumbs.append(
37                 {
38                     "title": title,
39                     "summary": summary,
40                     "url": url,
41                     "source": source_name if source_name else /
42                     "NewsAPI.org Fashion"
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/finance.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_finnhub_general_news():
7     """
8     Fetches general market news articles from Finnhub API.
9
10    Returns:
11        list: A list of dictionaries containing article details.
12    """
13    try:
14        url = getattr(settings, 'FINNHUB_API_URL', None)
15        if not url:
16            raise ValueError("FINNHUB_API_URL is not set in Django settings.")
17
18        response = requests.get(url, timeout=15)
19        response.raise_for_status()
20        data = response.json()
21
22        crumbs = []
23        # Finnhub general news returns a list of articles
24        for article in data:
25            title = article.get("headline")
26            url = article.get("url")
27            summary = article.get("summary")
28            source = article.get("source")
29            datetime_unix = article.get("datetime")
30
31            published_at_str = None
32            if datetime_unix:
33                published_at_str = datetime.fromtimestamp(
34                    datetime_unix, tz=timezone.utc).isoformat()
35
36            if not title or not url:
37                continue
38
39            crumbs.append({
40                "title": title,
41                "summary": summary,
42                "url": url
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/food.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_spoonacular_random_recipes(limit=5):
7     """
8     Fetches random recipes from Spoonacular API.
9
10    Args:
11        limit (int): The number of random recipes to fetch.
12
13    Returns:
14        list: A list of dictionaries containing recipe details.
15    """
16    crumbs = []
17    try:
18        if not settings.SPOONACULAR_API_KEY:
19            raise ValueError(
20                "SPOONACULAR_API_KEY is not set in Django settings.")
21        if not settings.SPOONACULAR_API_URL:
22            raise ValueError(
23                "SPOONACULAR_API_URL is not set in Django settings.")
24
25        # Spoonacular's random endpoint supports a 'number' parameter
26        url = f"{settings.SPOONACULAR_API_URL}&number={limit}"
27
28        response = requests.get(url, timeout=15)
29        response.raise_for_status()
30        data = response.json()
31
32        for recipe in data.get("recipes", []):
33            title = recipe.get("title")
34            url = recipe.get("sourceUrl")
35            # Combine summary fields for richer content
36            summary = (
37                f"Servings: {recipe.get('servings', 'N/A')}. "
38                f"Prep Time: {recipe.get('readyInMinutes', 'N/A')} mins. "
39                f"Cuisines: {' '.join(recipe.get('cuisines', []))}. "
40                f"Dietary: {' '.join(recipe.get('diets', []))}. "
41                f"Instructions: {recipe.get('instructions', '')[:500]}..."
42            )
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
1 import requests
2 import time
3 from django.conf import settings
4 from datetime import datetime, timezone
5
6
7 def fetch_lastfm_top_artists_bios(limit=10):
8     """
9     Fetches top artists from Last.fm and then retrieves their biographies.
10
11     Args:
12         limit (int): The maximum number of top artists to fetch.
13
14     Returns:
15         list: A list of dictionaries containing artist details
16             (title, summary, url, source, published_at).
17     """
18     crumbs = []
19     try:
20         if not settings.LASTFM_API_KEY:
21             raise ValueError("LASTFM_API_KEY is not set in Django settings.")
22         if not settings.LASTFM_BASE_URL:
23             raise ValueError("LASTFM_BASE_URL is not set in Django settings.")
24
25         # 1. Get top artists
26         top_artists_url = (
27             f"{settings.LASTFM_BASE_URL}?"
28             f"method=chart.gettopartists&api_key={settings.LASTFM_API_KEY}"
29             f"&format=json&limit={limit}"
30         )
31         response = requests.get(top_artists_url, timeout=10)
32         response.raise_for_status()
33         top_artists_data = response.json()
34
35         artists = top_artists_data.get("artists", []).get("artist", [])
36
37         # 2. For each artist, get their detailed info (including bio)
38         for artist in artists:
39             artist_name = artist.get("name")
40             artist_url = artist.get("url")
41
42             if not artist_name:
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/news.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_newsdata_world_news():
7     """
8     Fetches general world news headlines from NewsData.io API.
9
10    Returns:
11        list: A list of dictionaries containing article details.
12    """
13    try:
14        url = getattr(settings, 'NEWSDATA_WORLD_NEWS_URL', None)
15        if not url:
16            raise ValueError(
17                "NEWSDATA_WORLD_NEWS_URL is not set in Django settings.")
18
19        response = requests.get(url, timeout=15)
20        response.raise_for_status()
21        data = response.json()
22
23        crumbs = []
24        for article in data.get("results", []):
25            title = article.get("title")
26            url = article.get("link")
27            summary = article.get("description")
28            source = article.get("source_id")
29            published_at_str = article.get("pubDate")
30
31            if not title or not url:
32                continue
33
34            crumbs.append({
35                "title": title,
36                "summary": summary,
37                "url": url,
38                "source": source if source else "NewsData.io World News",
39                "published_at": published_at_str,
40            })
41        return crumbs
42    except requests.exceptions.RequestException as req_err:
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/plants.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_perennial_guides():
7     """
8     Fetches care guides from Perennial API and returns a list of dictionaries
9     containing article details for plants and gardening topics.
10    """
11    try:
12        url = settings.PERENNIAL_API_URL
13        perennial_guide_base_url = getattr(
14            settings, 'PERENNIAL_GUIDE_BASE_URL', 'https://perennial.com/guide/'
15        )
16
17        response = requests.get(url, timeout=10)
18        response.raise_for_status()
19        data = response.json()
20
21        crumbs = []
22        for guide in data.get("data", []):
23            if guide.get("care_guide"):
24                item_url = f'{perennial_guide_base_url}{guide.get("slug")}' if \
25                    guide.get('slug') else ""
26
27                published_at = None
28
29                crumbs.append({
30                    "title": guide.get("common_name", "Care Guide"),
31                    "summary": guide.get("care_guide", {}).get(
32                        "description", ""
33                    ),
34                    "url": item_url,
35                    "source": "Perennial",
36                    "published_at": published_at,
37                })
38        return crumbs
39    except requests.exceptions.RequestException as req_err:
40        print(f"Perennial API request error: {req_err}")
41        return []
42    except ValueError as iso_err:
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/sports.py

w: 1366 px h: 599 px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_thenewsapi_sports():
7     """
8     Fetches general sports news articles from TheNewsAPI.
9
10    Returns:
11        list: A list of dictionaries containing article details.
12    """
13    try:
14        url = getattr(settings, 'THENEWSAPI_SPORTS_URL', None)
15        if not url:
16            raise ValueError(
17                "THENEWSAPI_SPORTS_URL is not set in Django settings.")
18
19        response = requests.get(url, timeout=15)
20        response.raise_for_status()
21        data = response.json()
22
23        crumbs = []
24        for article in data.get("data", []):
25            title = article.get("title")
26            url = article.get("url")
27            summary = article.get("description")
28            source = article.get("source")
29            published_at_str = article.get("published_at")
30
31            if not title or not url:
32                continue
33
34            crumbs.append({
35                "title": title,
36                "summary": summary,
37                "url": url,
38                "source": source if source else "TheNewsAPI",
39                "published_at": published_at_str,
40            })
41        return crumbs
42    except requests.exceptions.RequestException as req_err:
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/technology.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 from django.conf import settings
3 from datetime import datetime, timezone
4
5
6 def fetch_mediastack_technology_news():
7     """
8     Fetches technology news articles from Mediastack API,
9     filtered to English language.
10
11     Returns:
12         list: A list of dictionaries containing article details.
13     """
14     crumbs = []
15     try:
16         url = getattr(settings, 'MEDIASTACK_TECHNOLOGY_URL', None)
17         if not url:
18             raise ValueError(
19                 "MEDIASTACK_TECHNOLOGY_URL is not set in Django settings.")
20
21         response = requests.get(url, timeout=15)
22         response.raise_for_status()
23         data = response.json()
24
25         # Mediastack returns results under the 'data' key
26         for article in data.get("data", []):
27             title = article.get("title")
28             url = article.get("url")
29             summary = article.get("description")
30             source = article.get("source")
31             published_at_str = article.get("published_at")
32
33             if not title or not url:
34                 continue
35
36             crumbs.append({
37                 "title": title,
38                 "summary": summary,
39                 "url": url,
40                 "source": source if source else "Mediastack Technology",
41                 "published_at": published_at_str,
42             })
```

Settings:



Results:

All clear, no errors found

pipeline/tasks/trivia.py

w: 1366px h: 599px



CI Python Linter

```
1 import requests
2 import html
3 from django.conf import settings
4 from datetime import datetime, timezone
5
6
7 def fetch_useless_facts():
8     """
9     Fetches a random useless fact.
10
11     Returns:
12         list: A list containing one dictionary with fact details.
13     """
14     crumbs = []
15     try:
16         url = getattr(settings, 'USELESS_FACTS_API_URL', None)
17         if not url:
18             raise ValueError(
19                 "USELESS_FACTS_API_URL is not set in Django settings.")
20
21         response = requests.get(url, timeout=10)
22         response.raise_for_status()
23         data = response.json()
24
25         fact_text = data.get("text")
26         fact_source = data.get("source")
27         fact_source_url = data.get("source_url")
28
29         if fact_text:
30             crumbs.append({
31                 "title": f"Useless Fact: {fact_text[:50]}...",
32                 "summary": fact_text,
33                 "url": fact_source_url,
34                 "source": fact_source if fact_source else "Useless Facts API",
35                 "published_at": None,
36             })
37     except requests.exceptions.RequestException as req_err:
38         print(f"Useless Facts fetch error: {req_err}")
39         return []
40     except ValueError as val_err:
41         print(f"Configuration error for Useless Facts: {val_err}")
```

Settings:



Results:

All clear, no errors found



Preferences App

preferences/forms.py

w: 1366px h: 599px



CI Python Linter

```
1 from django import forms
2 from django.utils import timezone
3
4 from subscriptions.models import UserSubscription
5 from .models import UserPreference, Topic
6
7
8 class UserPreferenceForm(forms.ModelForm):
9     """
10     Form for users to set their topic preferences,
11     with limits based on their subscription plan.
12     """
13     class Meta:
14         model = UserPreference
15         fields = ['topics']
16
17     def __init__(self, *args, **kwargs):
18         self.user = kwargs.pop('user')
19         super().__init__(*args, **kwargs)
20
21         self.selected_topic_ids = set()
22         if self.instance and self.instance.pk:
23             self.selected_topic_ids = set(self.instance.topics.values_list(
24                 'id', flat=True))
25
26         self.topic_limit = None
27
28     try:
29         subscription = self.user.subscriptions.filter(
30             active=True,
31             end_date_gte=timezone.now()
32         ).order_by('-end_date').first()
33         if subscription.plan.name.lower() == 'basic':
34             self.topic_limit = 2
35             self.fields['topics'].help_text = (
36                 f'You can select up to {self.topic_limit} topics with '
37                 'your Basic plan.'
38             )
39         else:
40             self.fields['topics'].help_text = (
41                 'select any topics you want.'
42             )
```

Settings:



Results:

All clear, no errors found

preferences/models.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.db import models
2 from django.conf import settings
3 from django.utils.text import slugify
4
5
6 class Topic(models.Model):
7     """
8     Model representing a topic that users can subscribe to.
9     Each topic has a name, a slug for URL purposes, a description, and
10     an image.
11     The slug is automatically generated from the name and is unique.
12     If a slug already exists, it appends a number to make it unique.
13     """
14     name = models.CharField(max_length=100, unique=True)
15     slug = models.SlugField(unique=True)
16     description = models.TextField(blank=True)
17     image = models.CharField(max_length=255, blank=True, null=True)
18
19     def save(self, *args, **kwargs):
20         if not self.slug:
21             self.slug = slugify(self.name)
22             original_slug = self.slug
23             counter = 1
24             while Topic.objects.filter(slug=self.slug).exists():
25                 self.slug = f"{original_slug}-{counter}"
26                 counter += 1
27             super().save(*args, **kwargs)
28
29     def __str__(self):
30         return self.name
31
32
33 class UserPreference(models.Model):
34     """
35     Model representing user preferences for topics.
36     Each user can have multiple topics they are interested in.
37     The user is linked to the Django user model.
38     The topics are stored as a many-to-many relationship.
39     """
40     user = models.ForeignKey(
41         settings.AUTH_USER_MODEL, on_delete=models.CASCADE
42     )
```

Settings:



Results:

All clear, no errors found

preferences/tests.py

w: 1366 px h: 599 px



CI Python Linter

```
1 # preferences/tests.py
2
3 import datetime
4 from django.test import TestCase, Client
5 from django.urls import reverse
6 from django.utils import timezone
7 from django.core.files.uploadedfile import SimpleUploadedFile
8
9 from accounts.models import CustomUser
10 from subscriptions.models import SubscriptionPlan, UsersSubscription
11 from .models import UserPreference, Topic
12 from .forms import UserPreferenceForm
13
14 # Define a minimal dummy image content for tests
15 DUMMY_IMAGE_CONTENT = (
16     b'GIF89a\x01\x00\x01\x00\x00\xff\x00\x00,'
17     b'\x00\x00\x00\x00\x01\x00\x01\x00\x00\x02\x02D\x01\x00;'
18 )
19
20
21 class UserPreferenceFormTest(TestCase):
22     """
23     Tests for the UserPreferenceForm.
24     """
25
26     def setUp(self):
27         self.user = CustomUser.objects.create_user(
28             username='formuser', email='form@example.com',
29             password='password123'
30         )
31         # Create Topics with dummy images
32         self.topic1 = Topic.objects.create(
33             name='Topic A',
34             image=SimpleUploadedFile(
35                 'topic_a.gif',
36                 DUMMY_IMAGE_CONTENT,
37                 content_type='image/gif'
38             )
39         )
40         self.topic2 = Topic.objects.create(
41             name='Topic B',
42             image=SimpleUploadedFile(
```

Settings:



Results:

All clear, no errors found

preferences/urls.py



CI Python Linter

w: 1366 px h: 599 px



```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('set/', views.set_preferences, name='set_preferences'),
6 ]
7
```

Settings:



Results:

All clear, no errors found

preferences/views.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.shortcuts import render, redirect
2 from django.contrib.auth.decorators import login_required
3 from django.utils.timezone import now
4
5 from subscriptions.models import UserSubscription
6 from .forms import UserPreferenceForm
7 from .models import UserPreference
8
9
10 @login_required
11 def set_preferences(request):
12     """
13     Allows a logged-in user to set their preferences.
14
15     Requires an active subscription; redirects to 'choose_plan' if not.
16     Handles both GET (display form) and POST (process form) requests.
17     """
18     # Get the latest valid subscription
19     user_subscription = UserSubscription.objects.filter(
20         user=request.user,
21         active=True,
22         end_date__gte=now()
23     ).order_by('-end_date').first()
24
25     if not user_subscription:
26         return redirect('choose_plan')
27
28     # Get or create preferences object
29     user_preference_obj, _ = UserPreference.objects.get_or_create(
30         user=request.user
31     )
32
33     # Handle form
34     if request.method == 'POST':
35         form = UserPreferenceForm(
36             request.POST,
37             instance=user_preference_obj,
38             user=request.user,
39         )
40         if form.is_valid():
41             form.save()
42             return redirect('home')
```

Settings:



Results:

All clear, no errors found



Subscriptions
App

subscriptions/email.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.template.loader import render_to_string
2 from django.core.mail import EmailMultiAlternatives
3 from django.conf import settings
4 from django.urls import reverse
5 from django.utils import timezone
6 from datetime import timedelta
7 from django.db.models import Q
8 from django.contrib.sites.models import Site
9
10 from .models import UserSubscription
11
12
13 def send_subscription_confirmation_email(user, subscription):
14     """
15     Sends a subscription confirmation email to the user.
16     """
17     subject = 'InfoCrumbs Subscription Confirmation!'
18     from_email = settings.DEFAULT_FROM_EMAIL
19     recipient_list = [user.email]
20
21     current_site = Site.objects.get_current()
22     site_domain = current_site.domain
23
24     context = {
25         'user': user,
26         'subscription': subscription,
27         'profile_url': f"{settings.SITE_URL}{reverse('account_profile')}",
28         'site_name': 'InfoCrumbs',
29         'site_domain': site_domain,
30     }
31
32     html_content = render_to_string(
33         'subscriptions/email/subscription_confirmation.html', context
34     )
35     text_content = render_to_string(
36         'subscriptions/email/subscription_confirmation.txt', context
37     )
38
39     msg = EmailMultiAlternatives(
40         subject,
41         text_content,
42         from_email
```

Settings:



Results:

All clear, no errors found

subscriptions/models.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.db import models
2 from django.conf import settings
3 from django.utils import timezone
4 from datetime import timedelta
5
6
7 class SubscriptionPlan(models.Model):
8     """
9     Represents a subscription plan with different options.
10    """
11    PLAN_CHOICES = [
12        ('basic', 'Basic'),
13        ('premium', 'Premium'),
14    ]
15    name = models.CharField(max_length=20, choices=PLAN_CHOICES, unique=True)
16    topic_limit = models.PositiveIntegerField(default=2)
17    price = models.DecimalField(max_digits=6, decimal_places=2, default=0.00)
18
19    def __str__(self):
20        return self.get_name_display()
21
22
23 class SubscriptionFrequency(models.Model):
24     """
25     Represents the frequency of subscription billing.
26    """
27    name = models.CharField(max_length=20, unique=True)
28    duration_days = models.PositiveIntegerField()
29    discount_percent = models.PositiveIntegerField(default=0)
30
31    def __str__(self):
32        return f'{self.name} ({self.discount_percent}% off)'
33
34
35 class UserSubscription(models.Model):
36     """
37     Represents a user's subscription to a plan with a specific frequency.
38    """
39    user = models.ForeignKey(
40        settings.AUTH_USER_MODEL,
41        on_delete=models.CASCADE,
42        related_name='subscriptions'
```

Settings:



Results:

All clear, no errors found

subscriptions/tests.py

w: 1366 px h: 599 px



CI Python Linter

```
1 from django.test import TestCase, Client
2 from django.urls import reverse
3 from django.utils import timezone
4 from datetime import datetime, timedelta, date
5 from unittest.mock import patch
6 from decimal import Decimal
7 from django.core import mail
8
9
10 from accounts.models import CustomUser
11 from .models import SubscriptionPlan, SubscriptionFrequency, UserSubscription
12 from .utils import calculate_subscription_price
13 from .email import send_subscription_confirmation_email
14 from .email import send_subscription_expiry_reminders
15
16
17 class SubscriptionModelsTest(TestCase):
18     """
19     Tests for the models in the subscriptions app.
20     """
21
22     def setUp(self):
23         """
24         Set up common data for model tests.
25         """
26         self.user = CustomUser.objects.create_user(
27             username='testuser',
28             email='test@example.com',
29             password='password123',
30             date_of_birth=date(1990, 1, 1)
31         )
32         self.basic_plan = SubscriptionPlan.objects.create(
33             name='basic', topic_limit=2, price=Decimal('10.00')
34         )
35         self.premium_plan = SubscriptionPlan.objects.create(
36             name='premium', topic_limit=100, price=Decimal('20.00')
37         )
38         self.monthly_freq = SubscriptionFrequency.objects.create(
39             name='monthly', duration_days=30, discount_percent=0
40         )
41         self.annual_freq = SubscriptionFrequency.objects.create(
42             name='annual', duration_days=365, discount_percent=10
```

Settings:



Results:

All clear, no errors found

subscriptions/urls.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('choose/', views.choose_plan, name='choose_plan'),
6     path('subscribe/<int:plan_id>', views.subscribe, name='subscribe'),
7     path('status/', views.subscription_status, name='subscription_status'),
8 ]
9
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
1 from .models import SubscriptionPlan, SubscriptionFrequency
2
3
4 def calculate_subscription_price(plan_name, duration_days):
5     """
6     Calculates the subscription price based on plan name and duration.
7     Assumes SubscriptionPlan has a 'price' field
8     representing its base cost (e.g., monthly).
9     """
10
11     try:
12         plan = SubscriptionPlan.objects.get(name=plan_name)
13     except SubscriptionPlan.DoesNotExist:
14         return None
15
16     try:
17         frequency = SubscriptionFrequency.objects.get(
18             duration_days=duration_days
19         )
20         discount_percent = frequency.discount_percent
21     except SubscriptionFrequency.DoesNotExist:
22         discount_percent = 0 # No discount if frequency not found
23
24     MONTH_DAYS = 30
25     number_of_billing_units = duration_days / MONTH_DAYS
26
27     raw_total_price = float(plan.price) * number_of_billing_units
28
29     discount_amount = raw_total_price * (discount_percent / 100)
30     final_price = raw_total_price - discount_amount
31
32     return round(final_price, 2)
```

Settings:



Results:

All clear, no errors found

subscriptions/views.py

w: 1366px h: 599px



CI Python Linter

```
1 from django.conf import settings
2 from django.shortcuts import render, redirect, get_object_or_404
3 from django.contrib.auth.decorators import login_required
4 from django.utils.timezone import now
5 from datetime import timedelta
6
7 from .models import SubscriptionPlan, UsersSubscription, SubscriptionFrequency
8 from .utils import calculate_subscription_price
9
10
11 @login_required
12 def choose_plan(request):
13     """
14     Displays available subscription plans and their frequencies.
15     """
16     plans = SubscriptionPlan.objects.all()
17     frequencies = SubscriptionFrequency.objects.all()
18
19     plan_options = []
20
21     for plan in plans:
22         frequency_options = []
23
24         for freq in frequencies:
25             price = calculate_subscription_price(plan.name, freq.duration_days)
26
27             if price is not None:
28                 frequency_options.append({
29                     'id': freq.id,
30                     'name': freq.name,
31                     'discount': freq.discount_percent,
32                     'price': price,
33                 })
34             else:
35                 print(
36                     f"Warning: Could't calculate price for plan '{plan.name}'"
37                     f" and duration '{freq.duration_days}'"
38                 )
39
40         plan_options.append({
41             'id': plan.id,
42             'name': plan.get_name_display()
43         })
```

Settings:



Results:

All clear, no errors found



Subscriptions App

Management Command
folder



CI Python Linter

w: 1366 px h: 599 px

```
1 from django.core.management.base import BaseCommand
2 from subscriptions.email import send_subscription_expiry_reminders
3
4
5 class Command(BaseCommand):
6     help = "Sends subscription expiry reminders"
7
8     def handle(self, *args, **options):
9         send_subscription_expiry_reminders()
10
```

Settings:



Results:

All clear, no errors found